

中山大學
SUN YAT-SEN UNIVERSITY

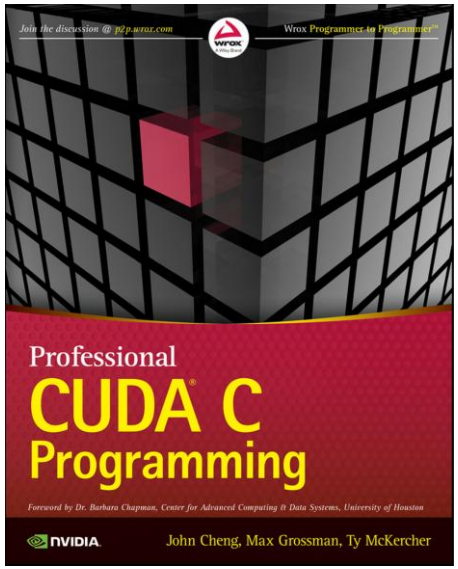


国家超级计算广州中心
NATIONAL SUPERCOMPUTER CENTER IN GUANGZHOU

CUDA简介

任课教师：吴迪

参考教材



- [Cheng] J. Cheng, M. Grossman, and T. McKercher, Professional CUDA C Programming, Wrox, 2014.

- 中译版：CUDA C编程权威指南
  - PDF: https://www.cs.utexas.edu/~rossbach/cs380p/papers/cuda-programming.pdf
  - https://github.com/deeperlearning/professional-cuda-c-programming

- **CUDA Programming Guide** 【Nvidia官方，最新版：2026.03.04】
  - https://docs.nvidia.com/cuda/cuda-programming-guide/

What is CUDA?



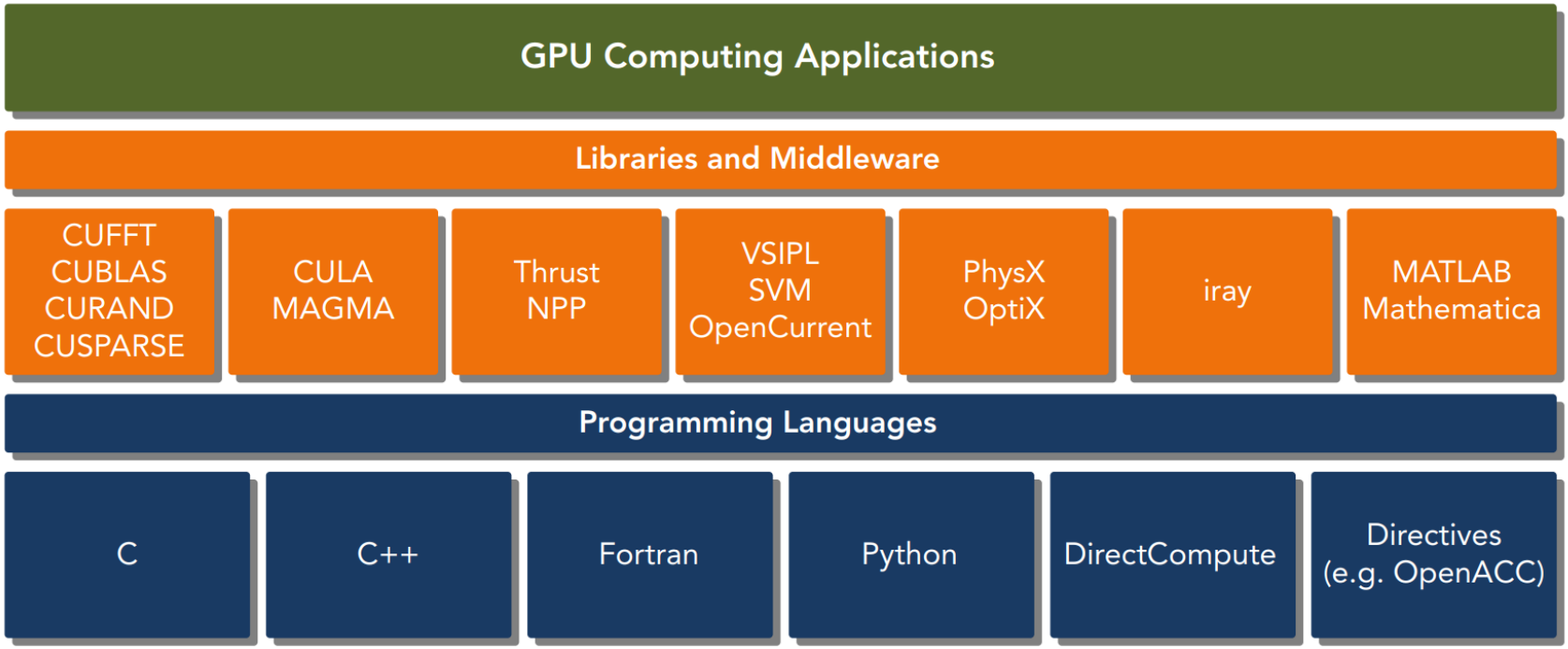
- **CUDA (Compute Unified Device Architecture)**
  - CUDA is a general-purpose parallel computing platform and programming model

- CUDA leverages the parallel compute engine in NVIDIA GPUs to solve many complex computational problems

- Using CUDA, you can access the GPU for computation, as done on the CPU.

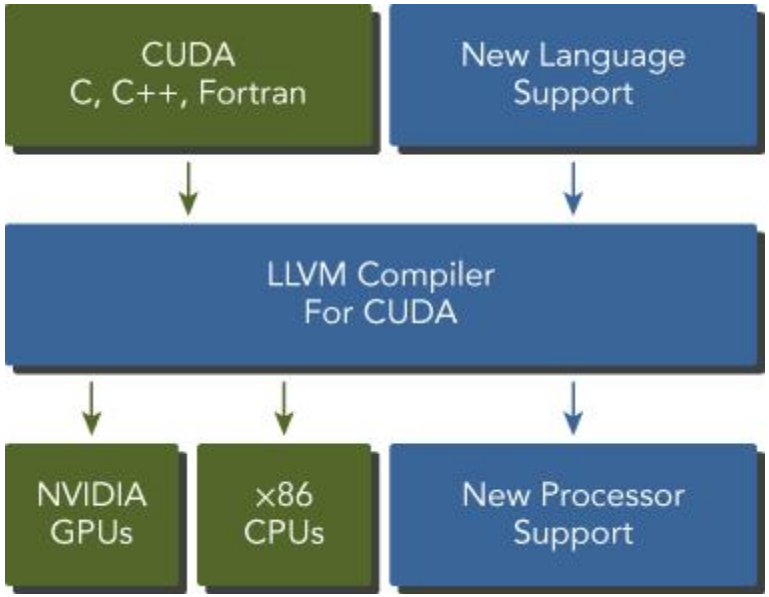
CUDA Platform

- The CUDA platform is accessible through:
  - CUDA-accelerated libraries, compiler directives, APIs,



CUDA C

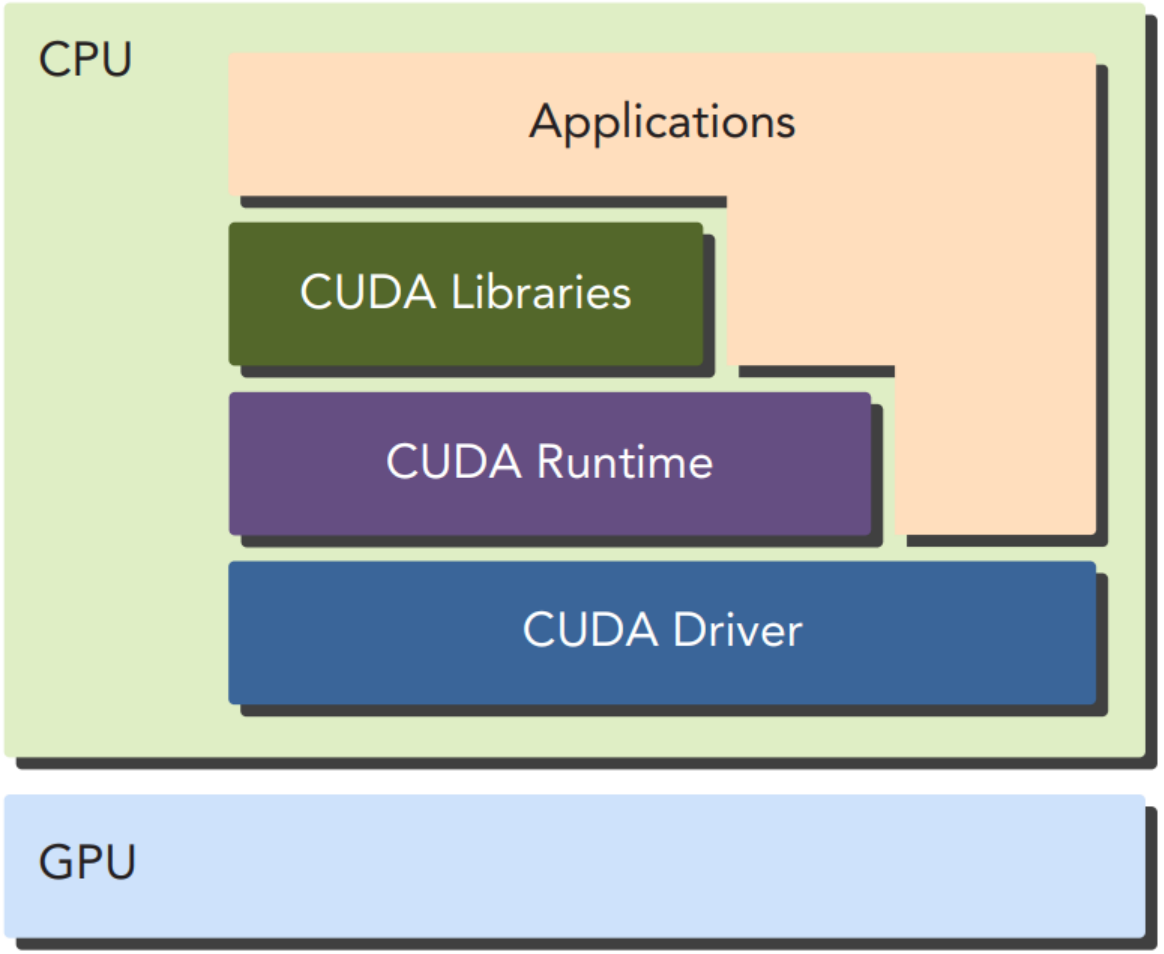
- **CUDA C is an extension of standard ANSI C**
  - with a handful of language extensions to enable heterogeneous programming, and
  - straightforward APIs to manage devices, memory, and other tasks.



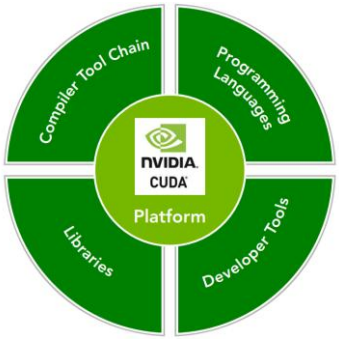
CUDA C

- CUDA provides two API levels for managing the GPU device and organizing threads:
  - **CUDA Driver API:** a low-level API and is relatively hard to program
    - but it provides more control over how the GPU device is used.
  - **CUDA Runtime API:** a higher-level API implemented on top of the driver API.
    - Each function of the runtime API is broken down into more basic operations issued to the driver API

CUDA Software Stack



CUDA Major Components



Compiler (bin/)

The CUDA-C and CUDA-C++ compiler, **nvcc**, is built on top of the NVVM optimizer, which is itself built on top of the LLVM compiler infrastructure. Developers who want to target NVVM directly can do so using the Compiler SDK, which is available in the **nvvm/** directory.

Tools (bin/)

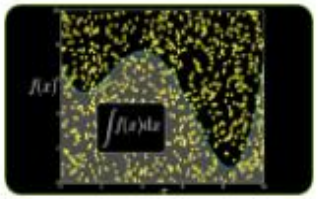
IDEs: nsight (Linux, Mac)
Debuggers: cuda-memcheck, cuda-gdb (Linux, Mac)
Profilers: nvprof, nvvp
Utilities: cuobjdump, nvdisasm, gwiz

Libraries (lib/)

- cublas (BLAS)
- cuda_occupancy (Kernel Occupancy Calculation [header file implementation])
- cudadevrt (CUDA Device Runtime)
- cudart (CUDA Runtime)
- cufft (Fast Fourier Transform [FFT])
- cupti (Profiling Tools Interface)
- curand (Random Number Generation)
- cusolver (Dense and Sparse Direct Linear Solvers and Eigen Solvers)
- cusparse (Sparse Matrix)
- npp (NVIDIA Performance Primitives [image and signal processing])
- nvcuvid (CUDA Video Decoder [Windows, Linux])
- nvgraph (CUDA nvGRAPH [accelerated graph analytics])
- nvml (NVIDIA Management Library)

CUDA Libraries & Tools

Libraries



cuRAND



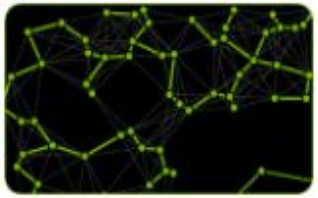
NPP



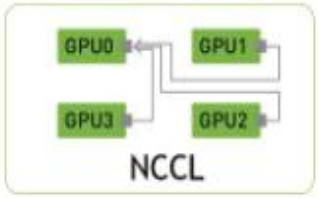
Math Library



cuFFT



nvGRAPH

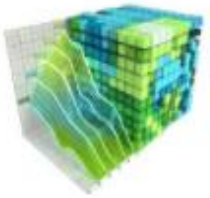


NCCL

Tools and Integrations



Nsight



Visual Profiler



CUDA GDB



CUDA MemCheck

OpenACC

OpenACC

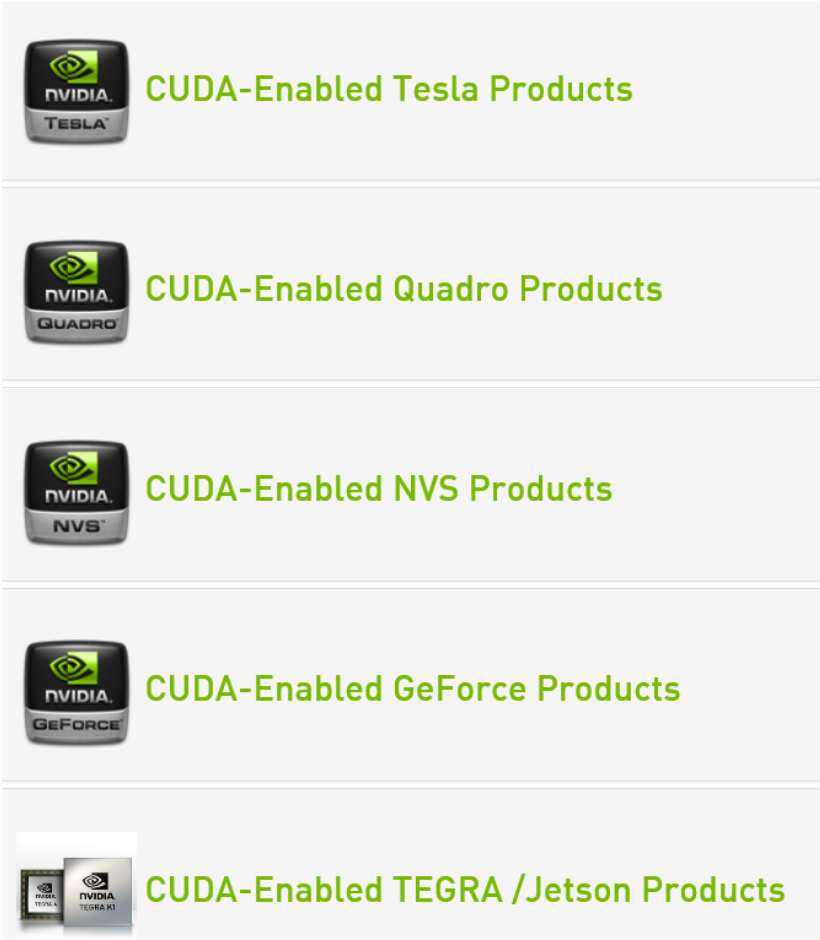


CUDA Profiling Tools

CUDA Installation (1)

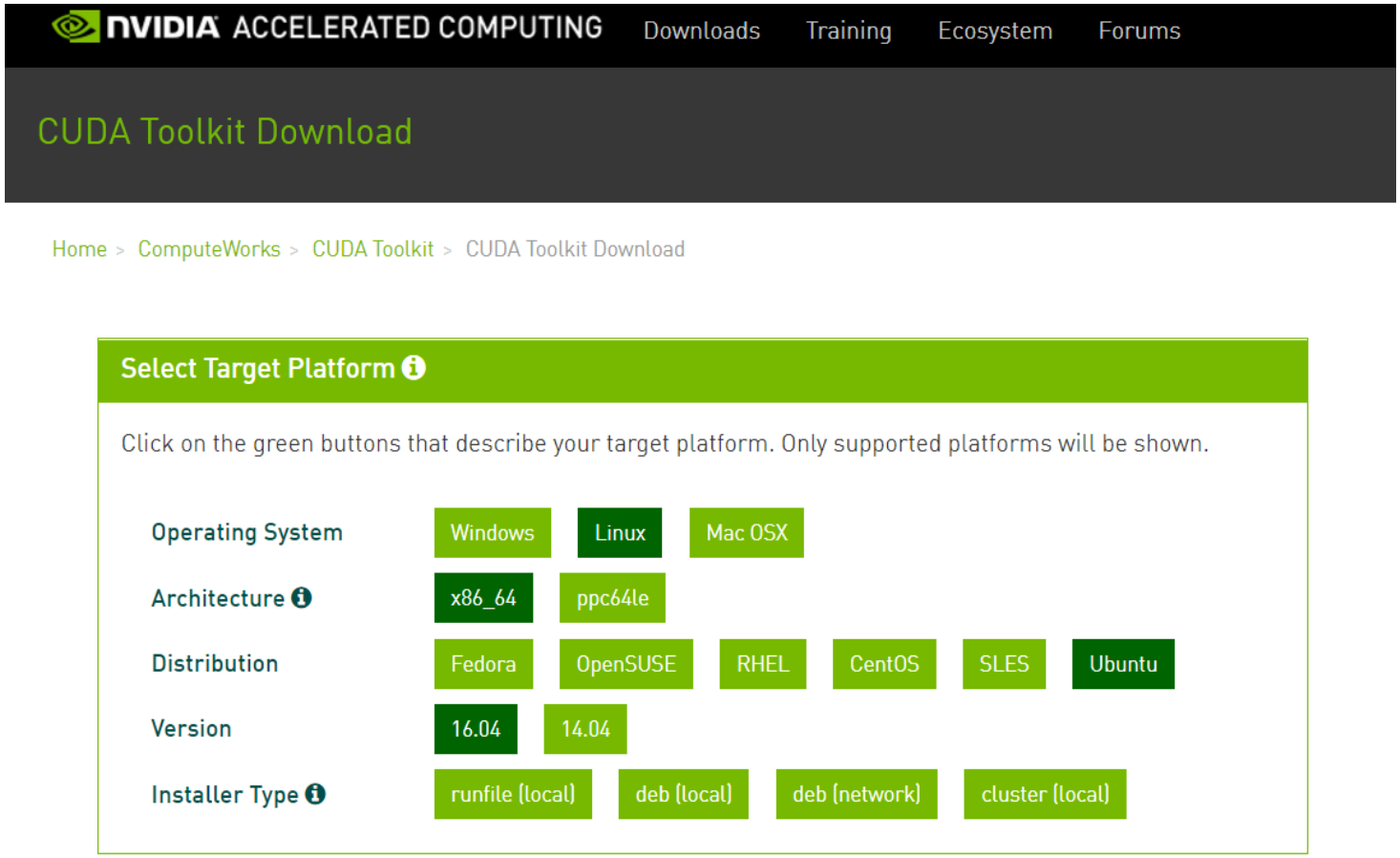
- Step 1: CUDA-capable GPU

- If your GPU is an NVIDIA card that is listed on the CUDA-supported GPUs page, your GPU is CUDA-capable



CUDA Installation (2)

- Step 2: Select platform



CUDA Installation (3)

- Step 3: Install CUDA

1. Install the repository meta-data, update the apt-get cache, and install CUDA:

```
$ sudo dpkg --install cuda-repo-<distro>-<version>.<architecture>.deb
$ sudo apt-get update
$ sudo apt-get install cuda
```

2. Reboot the system to load the NVIDIA drivers.
3. Set up the development environment by modifying the PATH and LD_LIBRARY_PATH variables:

```
$ export PATH=/usr/local/cuda-8.0/bin${PATH:+:${PATH}}
$ export LD_LIBRARY_PATH=/usr/local/cuda-8.0/lib64\
                         ${LD_LIBRARY_PATH:+:${LD_LIBRARY_PATH}}
```

4. Install a writable copy of the samples then build and run the nbody sample:

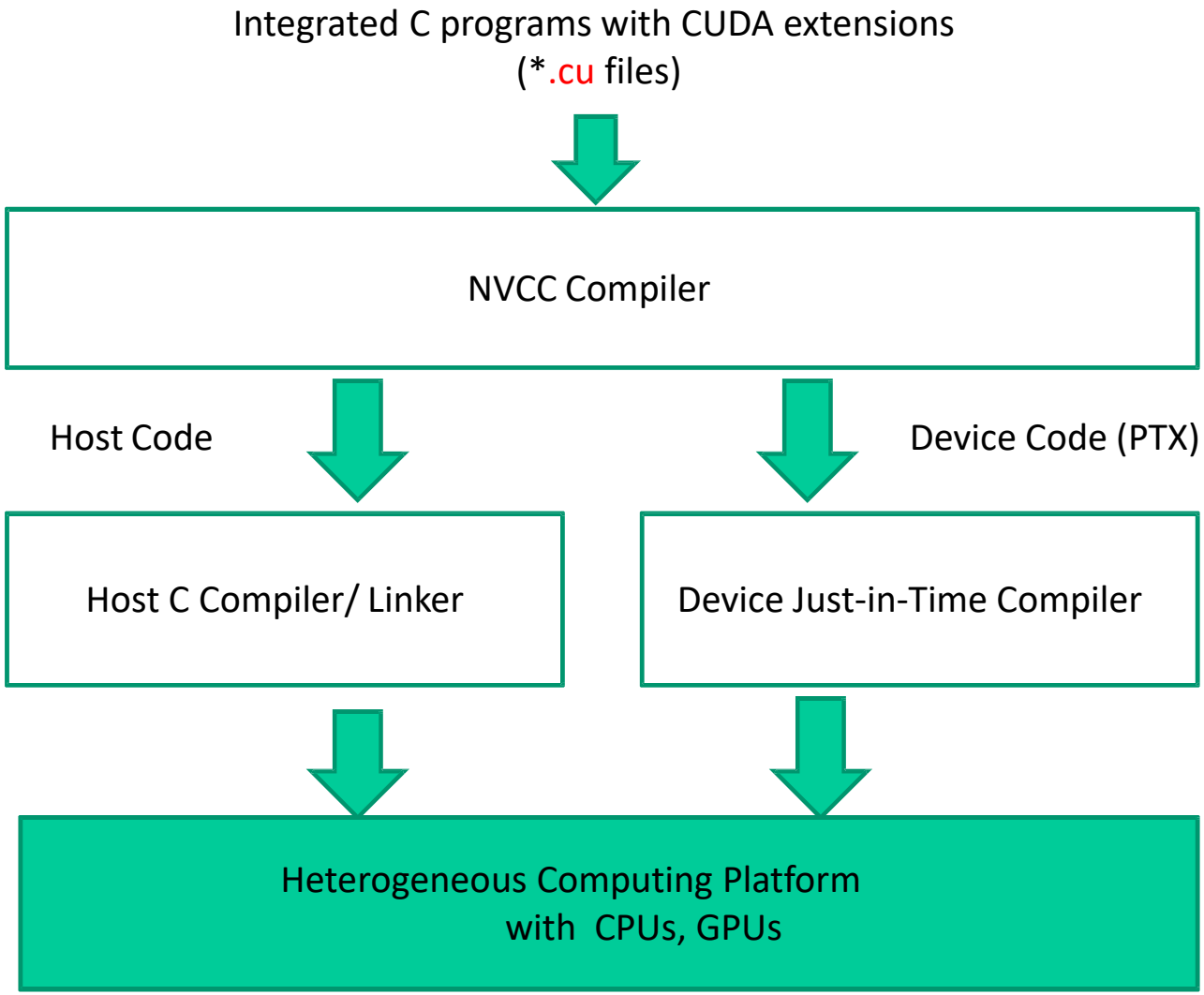
```
$ cuda-install-samples-8.0.sh ~
$ cd ~/NVIDIA_CUDA-8.0_Samples/5_Simulations/nbody
$ make
$ ./nbody
```

Hello World from GPU

How to Write a CUDA C Program?

- 1. Create a source code file with the special file name extension of .cu.
- 2. Compile the program using the CUDA nvcc compiler.
- 3. Run the executable file from the command line, which contains the kernel code executable on the GPU.

CUDA Compiler



Hello World from CPU

```
#include <stdio.h>
int main(void){
      printf("Hello World from CPU!\n");
      return 0;
}
```

- Standard C that runs on the host
- NVIDIA compiler (nvcc) can be used to compile programs with no *device* code

- `$ nvcc hello.cu -o hello`
- `$ ./hello`
  - `Hello World from CPU!`

Hello World from GPU (hello.cu)

```
#include <stdio.h>

__global__ void helloFromGPU (void)
{
      printf( "Hello World from GPU!\n" );
}

int main(void)
{
      // hello from cpu
      printf( "Hello World from CPU!\n" );

      helloFromGPU <<<1, 10>>>();

      cudaDeviceReset();
      return 0;
}
```

tells the compiler that the function will be called from the CPU and executed on the GPU

Launch the kernel function and execute 10 GPU threads

destroy and clean up all resources associated with the current device in the current process

Hello World from GPU (hello.cu)

- ```
  $ ./hello
  Hello World from CPU!
  Hello World from GPU!
  Hello World from GPU!
  Hello World from GPU!
  Hello World from GPU!
  Hello World from GPU!
  Hello World from GPU!
  Hello World from GPU!
  Hello World from GPU!
  Hello World from GPU!
  Hello World from GPU!
  ```



10 times
  ```

Is CUDA C Programming Difficult?

- Main difference between CPU programming and GPU programming is the level of programmer exposure to GPU architectural features.
- Need to think in parallel and have a basic understanding of GPU architecture
- To write efficient code as a parallel programmer, you need a basic knowledge of CPU architectures.
  - Temporal locality
  - Spatial locality

Is CUDA C Programming Difficult?

- Pthreads/OpenMP:
  - need to explicitly organize your threads

- CUDA C:
  - just write a piece of serial code to be called by only one thread
  - The GPU takes this kernel and makes it parallel by launching thousands of threads, all performing that same computation.
  - CUDA abstracts away the hardware details and does not require applications to be mapped to traditional graphics APIs.

Concluding Remarks

- CPU + GPU systems have become mainstream in the high-performance computing world.

- The data-parallel workload is executed on the GPU, while the serial and task-parallel workload is executed on the CPU.